GUÍA TÉCNICA DE ARMADO E IMPLEMENTACIÓN — tesis-iot-unsa

Proyecto: Prototipo IoT híbrido para evaluación de QoS (Wi-Fi 2,4 GHz + MQTT) con diseño factorial 3×3×3 y modelado predictivo. **Alcance del documento:** ciclo de vida completo del experimento — de los pines del ESP32 a las métricas del modelo de Machine Learning. **Relación con la tesis:** este documento es el complemento operativo de `metodologia.md` (apartados 3.1.5, 3.4 y 3.4.5). Toda decisión técnica que aquí se fija debe quedar registrada en el cuaderno de campo antes de iniciar la campaña.

Índice

1. Fase 1 — Armado de hardware y conexión física
2. Fase 2 — Configuración de red y broker MQTT
3. Fase 3 — Código de firmware (ESP32)
4. Fase 4 — Adquisición y almacenamiento de datos
5. Fase 5 — Análisis con Machine Learning
6. Lista de verificación global

Fase 1 — Armado de hardware y conexión física

1.1 Lista de materiales

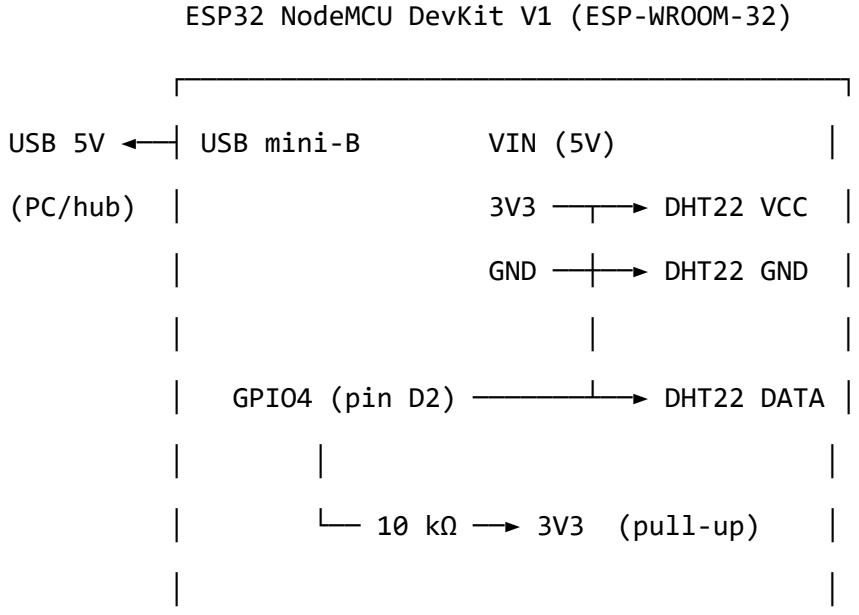
#	Componente	Cantidad	Especificación / nota
1	Placa ESP32 NodeMCU DevKit V1	4	Chip **ESP-WROOM-32**, USB mini-B, regulador 3,3 V integrado. 3 nodos emisores (N1, N2, N3) + 1 nodo suscriptor (S)
2	Sensor DHT22 (AM2302)	4	Temperatura/humedad como covariable ambiental de control (opcional pero recomendado)
3	Resistencia 10 kΩ	4	*Pull-up* del data del DHT22
4	Cable USB mini-B	4	Alimentación + programación (≥ 1 A por puerto)
5	Hub USB o fuentes 5 V dedicadas	2–4	Evitar compartir hub de baja capacidad (pico de corriente del ESP32 ≈ 500 mA)
6	Router Wi-Fi doméstico	1	2,4 GHz, 802.11 b/g/n, soporta canal fijo y ancho de 20 MHz

#	Componente	Cantidad	Especificación / nota
7	PC estación de adquisición	1	Conexión **cableada** Ethernet al router (ver §2.1)
8	Cable Ethernet Cat5e/Cat6	1	Enlace PC–router
9	Cinta métrica, cinta adhesiva, impresiones del croquis	—	Fijación de posiciones sin desplazamientos intra-ensayo

Criterio de selección: costo por nodo compatible con despliegues masivos (capa 1 de la propuesta). El RSSI/SNR se obtiene del propio controlador — no se requiere analizador de espectro.

1.2 Diagrama de pines (wiring)

Cada nodo (emisor o suscriptor) se arma idéntico, salvo el `NODE_ID` definido en firmware:



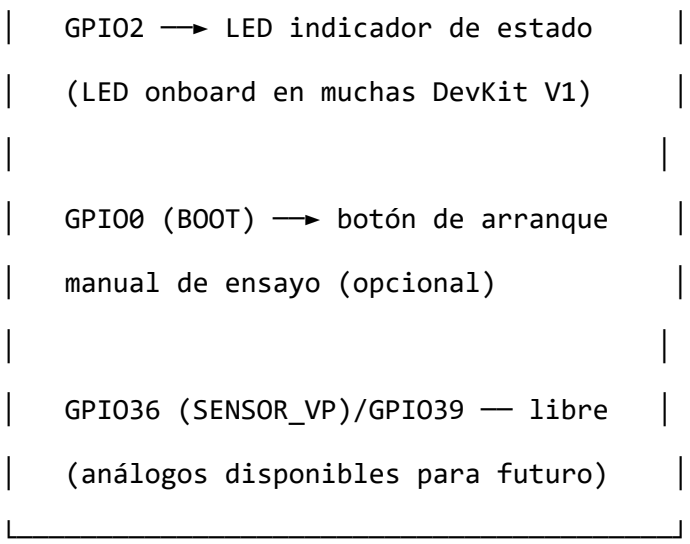


Tabla de conexiones

Señal	Pin ESP32	Observación
DHT22 DATA	**GPIO4**	Con *pull-up* 10 kΩ a 3V3, cable ≤ 20 cm
DHT22 VCC	**3V3**	Sensor tolera 3,3–5,5 V; a 3V3 evita nivel shifter
DHT22 GND	**GND**	Común con la placa
LED estado	**GPIO2**	LED onboard; parpadea al publicar, sólido al conectado
Botón inicio (opcional)	**GPIO0**	BOOT; no mantener pulsado al energizar
Alimentación	**USB mini-B (5V)**	Por cable al hub/PC; **no** alimentar desde 3V3 externo
Nodo suscriptor	Igual	Sin cambios de hardware; solo cambia firmware

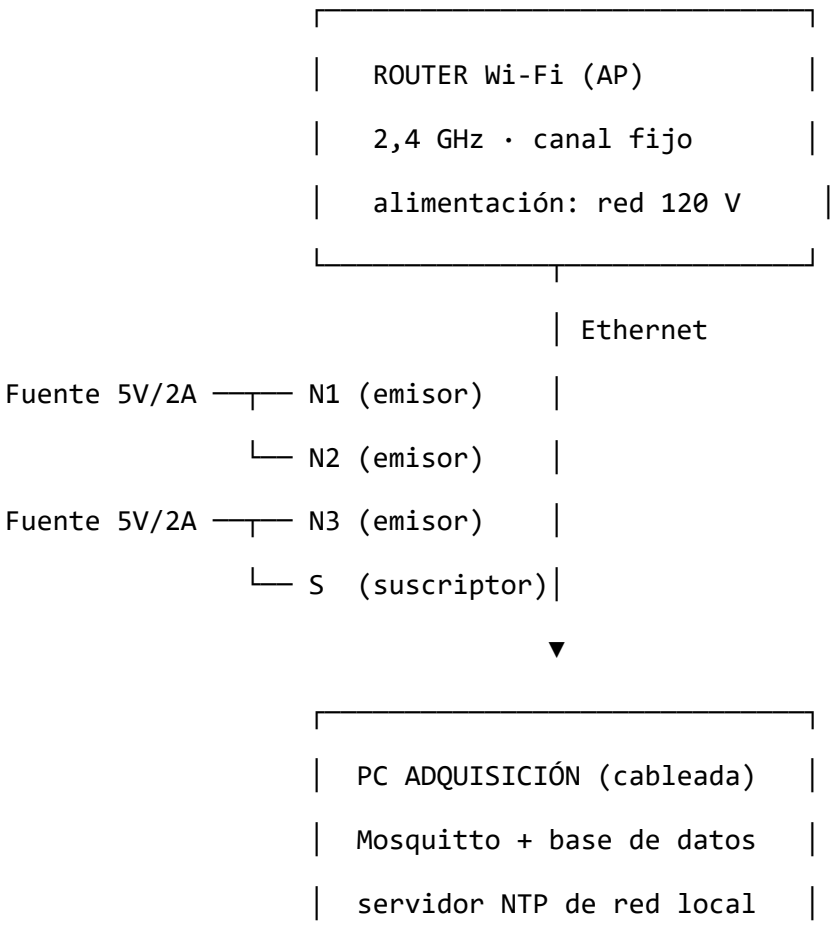
Notas de cableado

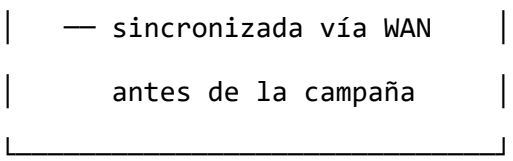
1. **Tierra común:** todos los GND derivan del USB; no crear tierras separadas entre sensor y

placa.

2. **Longitud DHT22:** > 20 cm sin *pull-up* adecuado produce lecturas erráticas; si se necesita sensor remoto, subir a 4,7 kΩ y blindar el cable.
3. **Alimentación estable:** el ESP32 en transmisión W-Fi alcanza picos de ~500 mA; un hub USB de 100 mA por puerto provoca reinicios aleatorios que anulan el ensayo (condición de parada tipo iii del protocolo). Priorizar **una fuente 5 V/1 A por dos placas** o puertos USB directos de la PC.
4. **Verificación de encendido:** al energizar, el LED debe encender y el monitor serie mostrar el banner de firmware (§3.4). Reinicio espontáneo durante el banner = problema de alimentación.

1.3 Esquema de alimentación del escenario

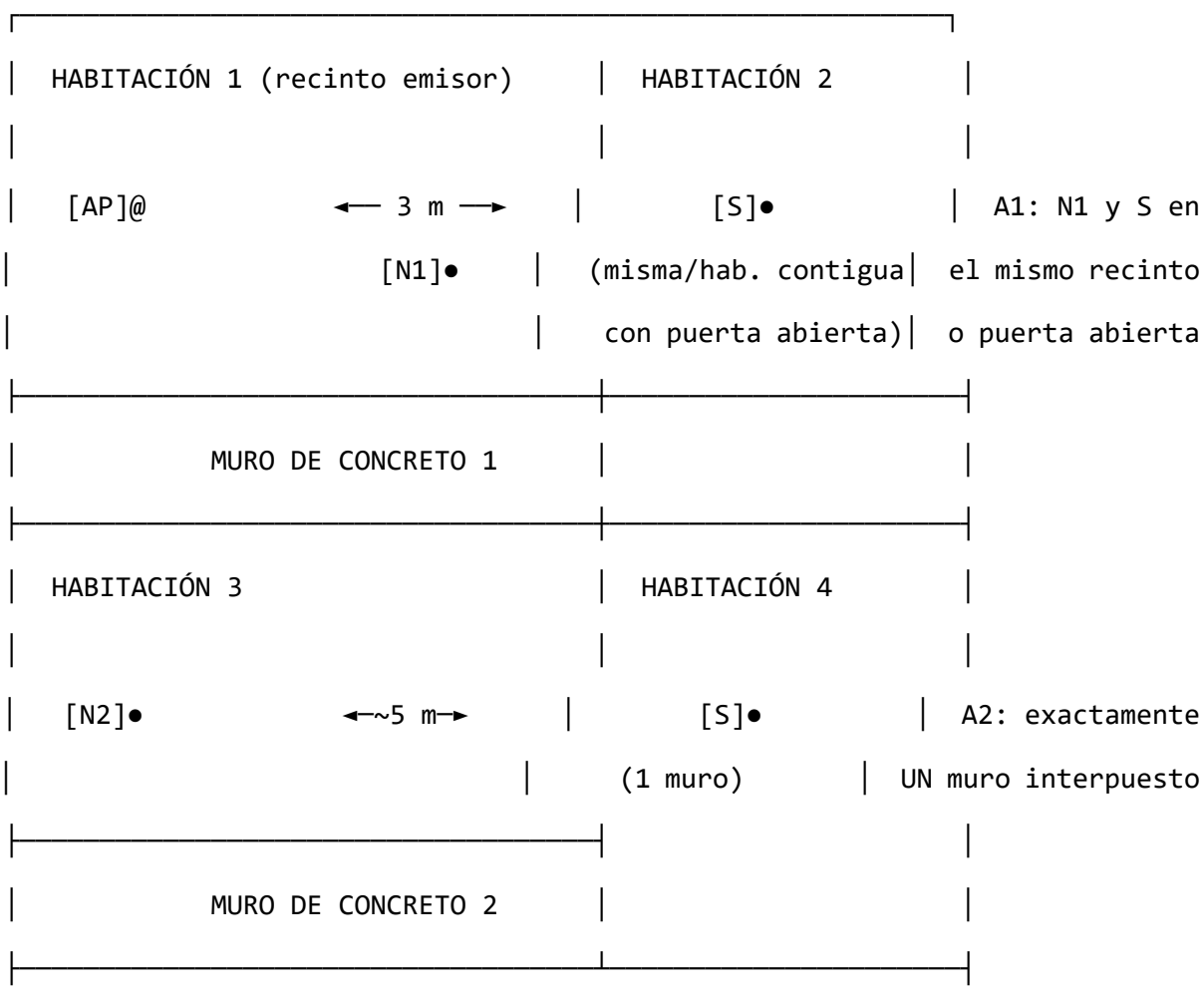


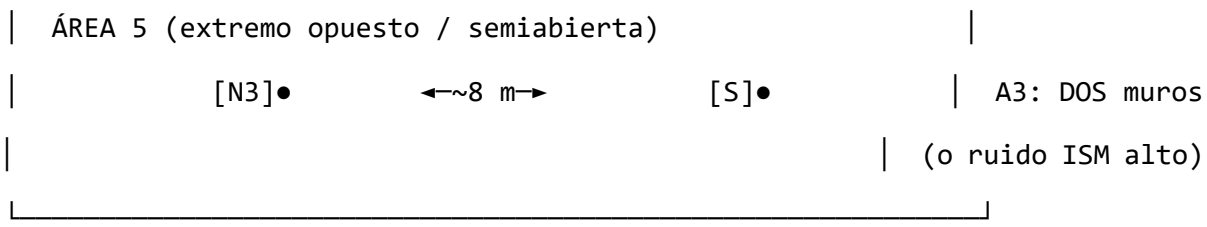


- Cada par de placas se alimenta de una fuente 5 V dedicada; nunca encadenar más de 2 placas por hub pasivo.
- Todo el equipo (AP, placas, PC) toma corriente de la misma instalación eléctrica de la vivienda → cero deriva de tierra entre nodos.
- **Regla:** dentro de una celda no se desconecta, reinicia ni reubica ningún equipo (§3.1.5).

1.4 Escenario residencial: niveles A1, A2 y A3

Croquis de referencia (medir y anotar distancias reales en `geometría.csv`, §4.2):





[N1|N2|N3]● = nodo emisor [S]● = suscriptor @ = punto de acceso (AP)

El AP y la PC NO se desplazan durante toda la campaña.

Operacionalización de A (copiar al cuaderno de campo):

Nivel	Condición	Criterio verificable antes del ensayo
A1	Visibilidad directa	Línea visual $N_i \leftrightarrow S$; sin muro de concreto; distancia métrica corta
A2	Un muro	Exactamente **1** muro de concreto en el trayecto; distancia comparable a A1
A3	Doble muro o ruido ISM	**2** muros de concreto **o** ruido de fondo ≥ umbral fijado en calibración; variante usada se anota

Checklist Fase 1

☐ 4 placas encienden sin reinicios en su fuente definitiva (prueba de 15 min).

☐ DHT22 responde en los 4 nodos (lectura serial).

☐ Croquis con distancias medidas; `geometría.csv` poblado.

☐ Fotos de cada posición N_i/S por nivel A (anexo del documento final).

☐ ID de placa físico etiquetado con cinta: `N1`, `N2`, `N3`, `S`.

Fase 2 — Configuración de red y broker MQTT

2.1 Router Wi-Fi: canal fijo en 2,4 GHz

Objetivo: el medio de contienda (capa 2) debe ser **constante** a lo largo de toda la campaña; un canal que cambie por auto-selección invalida la celda.

Pasos:

1. **Análisis previo:** desde la PC o con el propio ESP32 (script `scan`, §3.6), levantar la lista de redes vecinas y su RSSI en 2,4 GHz. Elegir entre **1, 6 u 11** (canales no solapados de 20 MHz) el de menor energía vecina. Registrar resultado y hora en el cuaderno.
2. **Fijar el canal:** panel del router → *Wireless 2,4 GHz*:
   - Canal: **el elegido (1/6/11)** — desactivar *Auto* / *Dynamic*.
   - Ancho de canal: **20 MHz** (40 MHz amplía el ancho pero eleva la susceptibilidad a interferencia; se mantiene 20 MHz como condición de control).
   - Modo: 802.11 b/g/n mixto.
   - SSID: fijo (ej. `TESIS-IOT-2G`), **visible**, sin ocultar.
   - Seguridad: WPA2-PSK (AES); anotar clave en el cuaderno.
   - **Desactivar:** *band steering*, *Smart Connect*, *auto-channel*, *WMM power save* si existe.
   - 5 GHz: **deshabilitar** el SSID (o renombrarlo para que ningún nodo lo use).
3. **IPs fijas:** reservar DHCP para la PC adquisición (`192.168.1.100`) — es la dirección del broker (§2.2).
4. **Aislamiento WAN:** según §3.1.2, el router opera **sin enlace a Internet** durante la campaña principal (WAN desconectada o *disable*). La sincronización NTP se resuelve con servidor local (§2.5). Solo los ensayos remotos complementarios habilitan WAN.
5. **Captura de configuración:** foto del panel con canal/SSID/potencia → anexo. Antes de **cada ensayo** se verifica que el canal no haya cambiado tras un reinicio (checklist §4.4).

¿Por qué la PC cableada? Si la PC se conecta por Wi-Fi, su tráfico de suscripción compite en el mismo medio y contamina la PDR/latencia medida. Cable Ethernet = solo los cuatro ESP32 disputan el aire.

2.2 Instalación y puesta en marcha del broker Mosquitto

En Windows (PC de adquisición):

```
# 1. Descargar el instalador oficial: https://mosquitto.org/download/
#    Durante la instalación: marcar "mosquitto como servicio de Windows"
#    y las herramientas de línea de comandos (mosquitto_sub/pub).

# 2. Verificar servicio
Get-Service mosquitto
Start-Service mosquitto
```

En Debian/Ubuntu:

```
sudo apt update && sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
```

Archivo de configuración (`mosquitto.conf` — reemplazar contenido; la seguridad se resuelve por capa de red, no por anonimato en Internet):

```
# Escucha solo en la LAN del escenario experimental
listener 1883 0.0.0.0
allow_anonymous true

# Persistencia y logging (requisito de triangulación §3.4.3)
persistence true
persistence_location C:/mosquitto/data/      # en Linux: /var/lib/mosquitto/
autosave_interval 60
```

```
log_dest file C:/mosquitto/log/mosquitto.log # en Linux: /var/log/mosquitto.log
log_type error
log_type warning
log_type notice
log_type information
log_timestamp true
log_timestamp_format %Y-%m-%dT%H:%M:%S.%Q     # marcas de tiempo con ms
```

Aislamiento: en el firewall de Windows permitir el puerto 1883 **solo** para el subred `192.168.1.0/24`; no habilitar reenvío de puertos ni UPnP. El broker nunca es visible desde Internet (la WAN está caída, §2.1.4).

Prueba de humo:

```
# Terminal 1 (escucha)
mosquitto_sub -h 127.0.0.1 -p 1883 -t "tesis/#" -v

# Terminal 2 (publicación)
mosquitto_pub -h 127.0.0.1 -p 1883 -t "tesis/prueba" -m "hola" -q 1

# Esperado en terminal 1:
# tesis/prueba hola
```

Arranque como servicio de larga duración (la campaña corre 81 ensayos; el broker no se toca a mitad):

```
Restart-Service mosquitto   # solo en ventanas entre ensayos
```

2.3 Estructura de tópicos MQTT

Convención: `tesis/<dominio>/<entidad>/<asunto>` — en minúsculas, sin espacios.

```
tesis/
├── control/
```

```
│   ├── config        # PC → nodos: parámetros de celda (B y C)  [QoS 1, retenido]
│   └── orden         # PC → nodos: inicio/parada del ensayo     [QoS 1, retenido]
├── nodos/
│   ├── N1/
│   │   ├── telemetria  # N1 → broker: paquete de datos          [QoS = nivel C ensayado]
│   │   └── estado      # N1 → broker: online/offline (LWT)      [QoS 0, retenido]
│   ├── N2/
│   │   ├── telemetria
│   │   └── estado
│   └── N3/
│       ├── telemetria
│       └── estado
└── suscriptor/
    ├── registros       # S → broker: registro enriquecido con latencia [QoS 1]
    └── estado          # S → broker: LWT retenido
```

Reglas semánticas:

Tópico	Carga	Notas
`tesis/control/config`	`b=1000;qos=1;c=A2B3C2;rep=2`	Fija intervalo del factor **B**, nivel **QoS (C)**, etiqueta de celda y réplica. Cada nodo lo confirma imprimiendo `[CONFIG] b=… qos=… celda=…` en la consola serie
`tesis/control/orden`	`iniciar` / `parar`	Da la orden de publicar; el cronómetro del ensayo arranca con el primer mensaje

Tópico	Carga	Notas
`tesis/nodos/Nx/telemetria`	JSON del §3.3	**El QoS de publicación de este tópico ES el factor C** (0/1/2)
`tesis/suscriptor/registros`	JSON del §4.1	Siempre QoS 1: es bookkeeping, no forma parte del factor
`*/estado`	`on`/`off`	Retenido + LWT: diagnóstico rápido de caídas (condición de parada)

Pérdida de paquetes y PDR no viajan como tópico: se derivan en la PC a partir de gaps de `seq` (emisor ↔ suscriptor ↔ log del broker), conforme al registro triángulo de §3.4.3.

2.4 QoS MQTT: correspondencia con los niveles C

Nivel ensayado	QoS MQTT	Semántica medida en cada mensaje
C1	0	*Fire and forget*: sin ACK → menor retardo, pérdida no gestionada
C2	1	PUBLISH ↔ PUBACK: al menos una entrega
C3	2	PUBLISH ↔ PUBREC ↔ PUBREL ↔ PUBCOMP: exactamente una entrega (4 manos)

El QoS se **cambia por tópico de control entre ensayos**, nunca recompilando firmware (idéntica versión de firmware en las 4 placas, §3.1.5).

2.5 Sincronización NTP sin Internet (requisito crítico)

La latencia de un solo sentido se calcula como `t_rx - t_tx` entre relojes distintos; una deriva de 30 ppm acumula ~18 ms en 10 minutos — mismo orden que la señal medida. Con WAN caída, el NTP público no está disponible:

- **Opción recomendada:** instalar un servidor NTP local en la PC (`Meinberg NTP Server` en Windows o `chrony` en Linux), apuntando su *upstream* al NTP público **solo durante la fase de preparación** (WAN arriba). Luego:

  ```
  configTime(0, 0, "192.168.1.100");  // en el firmware: NTP = IP de la PC
  ```

- **Verificación previa a cada ensayo:** los 4 nodos + PC reportan desfase ≤ ±10 ms (valor a declarar en el protocolo); si se supera → condición de parada (ii) §3.1.5.
- Registrar el desfase observado por sesión en el cuaderno (§3.4.3).

Checklist Fase 2

☐ Canal fijo 1/6/11 elegido con análisis de espectro; 20 MHz; SSID fijo; foto guardada.

☐ WAN del router deshabilitada; PC con IP `192.168.1.100` fija, cableada.

☐ `mosquitto` corriendo como servicio; log con timestamps ms creciendo.

☐ `mosquitto_sub -t "tesis/#"` recibe publicaciones de prueba.

☐ Prueba de los tres QoS (0/1/2) con `mosquitto_pub -q n` → entrega confirmada.

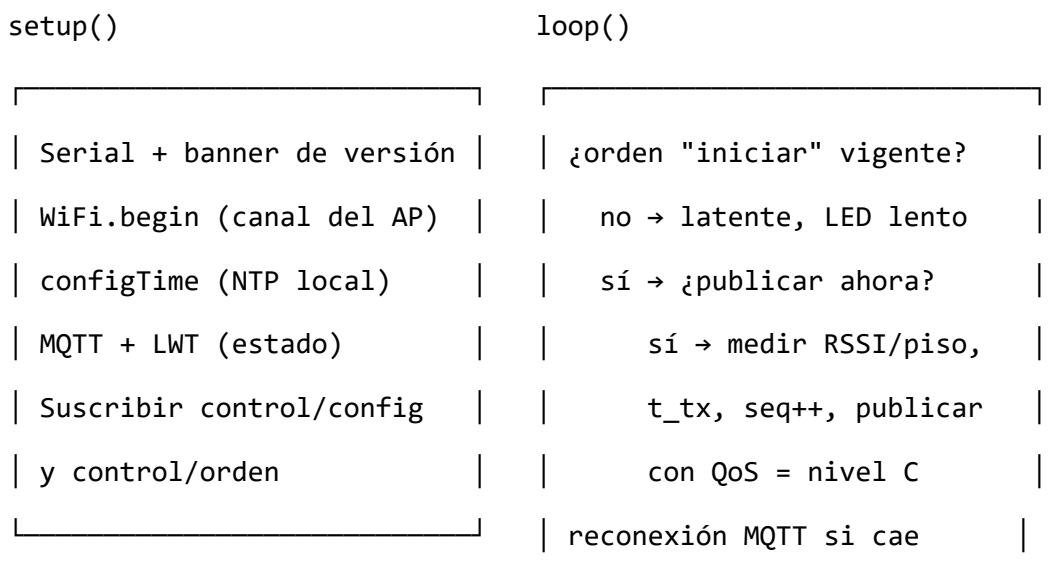
☐ Servidor NTP local respondiendo; desfase medido ≤ ±10 ms en los 4 nodos.

Fase 3 — Código de firmware (ESP32)

3.1 Entorno de desarrollo

- **Arduino IDE 2.x** (alternativa: ESP-IDF v5.x para quien requiera control fino; la guía entrega el código en C++/Arduino).
- Añadir gestor de placas: *Archivo* → *Preferencias* → *URLs de gestor adicional*: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- *Gestor de placas* → instalar **esp32 by Espressif Systems** → seleccionar **ESP32 Dev Module** (frecuencia 240 MHz, esquema de *flash* predeterminado, puerto serie correspondiente).
- Bibliotecas: **AsyncMqttClient** (Marcel Martin) + **AsyncTCP** (*Gestor de bibliotecas* → buscar `AsyncMqttClient` y `Async TCP`). Es el cliente MQTT de Arduino/ESP32 que permite **publicar con QoS 0, 1 y 2** — PubSubClient solo publica a QoS 0 y **no sirve para ensayar los niveles C2/C3**.
- Tamaño de payload: el JSON del §3.3 se mantiene < 256 bytes (carga útil constante); AsyncMqttClient no requiere ajustes de búfer.

3.2 Arquitectura del firmware



3.3 Código del nodo emisor (N1 / N2 / N3)

Guardar como `emisor/emisor.ino` (ajustar `NODE_ID`, SSID, clave y NTP antes de compilar):

```
// ============================================================
// NODO EMISOR — tesis-iot-unsa
// Publica telemetría periódica (RSSI, piso de ruido, timestamp)
// con intervalo (factor B) y QoS (factor C) configurables por MQTT.
// Librerías: AsyncMqttClient + AsyncTCP
// ============================================================
#include <WiFi.h>
#include <AsyncMqttClient.h>
#include <time.h>
#include <sys/time.h>

// ---------- Identidad y red (fijos por compilación) ----------
#define NODE_ID      "N1"              // N1 | N2 | N3  (cambiar por placa)
#define WIFI_SSID    "TESIS-IOT-2G"
#define WIFI_PASS    "clave-secreta"
#define MQTT_HOST    "192.168.1.100"   // PC de adquisición
#define MQTT_PORT    1883
#define NTP_SERVER   "192.168.1.100"   // servidor NTP local (§2.5)

// ---------- Niveles del factor B (fijar ANTES de la campaña) ----------
#define B1_MS 5000   // B1 oferta esparsa
#define B2_MS 1000   // B2 oferta moderada
#define B3_MS  250   // B3 oferta intensa
```

```
const int NOISE_FLOOR_DBM = -92;  // piso de ruido medido en calibración (§3.1.4)

AsyncMqttClient mqtt;
char estadoTopic[48], teleTopic[48];

int      intervalo_ms = B2_MS;     // reconfigurable por tesis/control/config
int      mqtt_qos     = 1;         // nivel C vigente (0/1/2)
bool     corriendo    = false;     // orden de ensayo activa
char     celda[12]    = "PEND";    // p.ej. "A2B3C2"
int      replica      = 0;
uint32_t seq = 0;
uint32_t proxima_pub = 0;

// ---------- Tiempo epoch en ms (reloj NTP) ----------
uint64_t epoch_ms() {
  struct timeval tv;
  gettimeofday(&tv, nullptr);
  return (uint64_t)tv.tv_sec * 1000ULL + (uint64_t)tv.tv_usec / 1000;
}

String mk(const char* p, size_t l) {  // payload -> String seguro (no null-terminated)
  String s; s.reserve(l);
  for (size_t i = 0; i < l; i++) s += p[i];
  return s;
}
```

```
// ---------- Callbacks MQTT ----------
void onMqttConnect(bool sessionPresent) {
  (void)sessionPresent;
  mqtt.publish(estadoTopic, 0, true, "on");
  mqtt.subscribe("tesis/control/config", 1);
  mqtt.subscribe("tesis/control/orden", 1);
  Serial.println("[MQTT] Conectado; suscrito a control (QoS 1)");
}

void onMqttDisconnect(AsyncMqttClientDisconnectReason reason) {
  Serial.printf("[MQTT] Desconectado (%d); reconecto en 1 s\n", (int)reason);
  delay(1000);
  mqtt.connect();
}

void onMqttMessage(char* topic, char* payload,
                   AsyncMqttClientMessageProperties props,
                   size_t len, size_t index, size_t total) {
  (void)props; (void)total;
  if (index != 0) return;              // mensajes de control: llegan en un trozo
  String msg = mk(payload, len);

  if (String(topic) == "tesis/control/orden") {
    corriendo = (msg.indexOf("iniciar") >= 0);
    if (corriendo) { seq = 0; proxima_pub = millis(); }
   Serial.printf("[ORDEN] %s -> %s\n", msg.c_str(), corriendo ? "CORRIENDO" : "DETENIDO");
    return;
```

```
  }
  if (String(topic) == "tesis/control/config") {
    int p;
   if ((p = msg.indexOf("b="))  >= 0) intervalo_ms = msg.substring(p + 2).toInt();
   if ((p = msg.indexOf("qos=")) >= 0) mqtt_qos = constrain(msg.substring(p + 4).toInt(), 0, 2)
    if ((p = msg.indexOf("c="))   >= 0) {
      String c = msg.substring(p + 2); c.replace(";", "");
      strncpy(celda, c.c_str(), sizeof(celda) - 1);
    }
    if ((p = msg.indexOf("rep=")) >= 0) replica = msg.substring(p + 4).toInt();
    Serial.printf("[CONFIG] b=%d qos=%d celda=%s rep=%d\n",
                  intervalo_ms, mqtt_qos, celda, replica);
  }
}

void setup() {
  Serial.begin(115200);
  Serial.printf("\n=== TESIS-IOT-UNSA EMISOR %s | fw 1.1 ===\n", NODE_ID);
  pinMode(2, OUTPUT);

  snprintf(estadoTopic, sizeof(estadoTopic), "tesis/nodos/%s/estado", NODE_ID);
 snprintf(teleTopic,  sizeof(teleTopic),  "tesis/nodos/%s/telemetria", NODE_ID);

  WiFi.mode(WIFI_STA);
 WiFi.setSleep(false);                 // power-save off: evita pausas de radio
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(250); Serial.print("."); }
```

```
  Serial.printf("\n[WiFi] OK  IP=%s  canal=%d  RSSI=%d dBm\n",
                WiFi.localIP().toString().c_str(), WiFi.channel(), WiFi.RSSI());

  configTime(0, 0, NTP_SERVER);           // reloj NTP de red local (§2.5)
  delay(1500);

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setClientId(NODE_ID);
  mqtt.setWill(estadoTopic, 0, true, "off");          // LWT: off si cae
  mqtt.onConnect(onMqttConnect);
  mqtt.onDisconnect(onMqttDisconnect);
  mqtt.onMessage(onMqttMessage);
  mqtt.connect();

  struct timeval tv; gettimeofday(&tv, nullptr);
  Serial.printf("[NTP] epoch=%llu ms\n",
                (unsigned long long)(tv.tv_sec * 1000ULL + tv.tv_usec / 1000));
}

void loop() {
  if (!mqtt.connected()) mqtt.connect();

  if (!corriendo) {                       // sin orden: latente, LED lento
    digitalWrite(2, (millis() / 1000) % 2);
    return;
  }
```

```
  uint32_t ahora = millis();
  if (ahora < proxima_pub) return;
  proxima_pub = ahora + intervalo_ms;

  // ---- Medición (§3.4.3: registro en el emisor) ----
  int      rssi = WiFi.RSSI();                    // índice de radiofrecuencia
  uint64_t t_tx = epoch_ms();                        // marca de emisión NTP
  seq++;

  char pay[224];                                 // tamaño constante en los 3 nodos
  snprintf(pay, sizeof(pay),
       "{\"n\":\"%s\",\"s\":%lu,\"tx\":%llu,\"r\":%d,\"nf\":%d,\"c\":\"%s\",\"rep\":%d}",
          NODE_ID, (unsigned long)seq, (unsigned long long)t_tx,
          rssi, NOISE_FLOOR_DBM, celda, replica);

  // QoS de publicación = nivel C ensayado (0/1/2) — AsyncMqttClient sí lo soporta
  uint16_t pkt = mqtt.publish(teleTopic, mqtt_qos, false, pay);
  if (pkt == 0) Serial.println("[PUB] FALLÓ (conexión o búfer)");

  digitalWrite(2, !digitalRead(2));
  if (seq % 100 == 0)
    Serial.printf("[PUB] seq=%lu rssi=%d q=%d pkt=%u\n",
                  (unsigned long)seq, rssi, mqtt_qos, pkt);
}
```

Notas técnicas del emisor:

1. **Payload constante:** el JSON es idéntico en tamaño en los tres nodos (§3.1.5: *tamaño de carga útil constante*); solo cambian `n`, `s`, `tx`, `r`.

2. **QoS efectivo (factor C):** `mqtt.publish(topico, mqtt_qos, false, pay)` devuelve el *packetId* (> 0 = aceptado por el broker). Para C2 el broker responde PUBACK y para C3 completa la secuencia PUBREC/PUBREL/PUBCOMP — verificable con `mosquitto_sub -t "tesis/#" -v` o con el `log_type information` del broker. Este es el motivo de usar AsyncMqttClient en lugar de PubSubClient.
3. **NTP:** `gettimeofday()` refleja el reloj sincronizado; si el servidor NTP no responde, `tx` queda en época 1970 → **parada inmediata** (condición ii §3.1.5). Validar en el banner: `epoch` > 1 700 000 000 000 ms.
4. **Piso de ruido (`nf`):** constante medida en calibración con app de espectro o `WiFi.scan()` en reposo; `snr = r - nf` se deriva en análisis (§5.1).

3.4 Código del nodo suscriptor (S)

```
// ============================================================
// NODO SUSCRIBOR — tesis-iot-unsa
// Recibe telemetría, calcula latencia extremo a extremo,
// verifica unicidad (seq) y republica el registro enriquecido.
// Librerías: AsyncMqttClient + AsyncTCP
// ============================================================
#include <WiFi.h>
#include <AsyncMqttClient.h>
#include <time.h>
#include <sys/time.h>

#define WIFI_SSID   "TESIS-IOT-2G"
#define WIFI_PASS   "clave-secreta"
#define MQTT_HOST   "192.168.1.100"
#define NTP_SERVER  "192.168.1.100"
```

```
AsyncMqttClient mqtt;

const char* IDS[3]  = {"N1", "N2", "N3"};
uint32_t    ultSeq[3] = {0, 0, 0};        // último seq aceptado por nodo
uint64_t    recibidos = 0, duplicados = 0;

uint64_t epoch_ms() {
  struct timeval tv; gettimeofday(&tv, nullptr);
  return (uint64_t)tv.tv_sec * 1000ULL + tv.tv_usec / 1000;
}

String mk(const char* p, size_t l) {
  String s; s.reserve(l);
  for (size_t i = 0; i < l; i++) s += p[i];
  return s;
}

int idxNodo(const String& n) { for (int i = 0; i < 3; i++) if (n == IDS[i]) return i; return -1; }

void onMqttConnect(bool) {
  mqtt.publish("tesis/suscriptor/estado", 0, true, "on");
  mqtt.subscribe("tesis/nodos/+/telemetria", 1);     // QoS 1 de suscripción
  Serial.println("[MQTT] Suscriptor conectado y suscrito");
}

void onMqttDisconnect(AsyncMqttClientDisconnectReason r) {
```

```
  Serial.printf("[MQTT] Desconectado (%d); reconecto\n", (int)r);
  delay(1000); mqtt.connect();
}

void onMqttMessage(char* topic, char* payload,
                   AsyncMqttClientMessageProperties props,
                   size_t len, size_t index, size_t total) {
  (void)topic; (void)props; (void)total;
  if (index != 0) return;
  String m = mk(payload, len);

  auto campo = [&m](const char* k) -> String {
    int a = m.indexOf(String("\"") + k + "\":"); if (a < 0) return "";
    a = m.indexOf(':', a) + 1;
    while (a < (int)m.length() && (m[a] == ' ' || m[a] == '"')) a++;
    int b = a;
   while (b < (int)m.length() && m[b] != '"' && m[b] != ',' && m[b] != '}') b++;
    return m.substring(a, b);
  };

  String  n  = campo("n");
  int     i  = idxNodo(n); if (i < 0) return;
  uint32_t s = (uint32_t)campo("s").toInt();
  if (s != 0 && s <= ultSeq[i]) { duplicados++; return; }   // unicidad (§3.4.3)
  ultSeq[i] = s;

  uint64_t tx  = strtoull(campo("tx").c_str(), nullptr, 10);
```

```
  uint64_t rx  = epoch_ms();                                  // recepción NTP
  long long lat = (long long)rx - (long long)tx;           // latencia 1 ida (ms)
  int rssi = campo("r").toInt();
  recibidos++;

  char reg[208];
  snprintf(reg, sizeof(reg),
       "{\"n\":\"%s\",\"s\":%lu,\"tx\":%llu,\"rx\":%llu,\"lat\":%lld,\"r\":%d,\"c\":\"%s\"
          n.c_str(), (unsigned long)s, (unsigned long long)tx,
          (unsigned long long)rx, lat, rssi, campo("c").c_str());
 mqtt.publish("tesis/suscriptor/registros", 1, false, reg);  // QoS 1 (bookkeeping)
 Serial.println(reg);                                  // copia serial de respaldo
}

void setup() {
  Serial.begin(115200);
  Serial.println("=== TESIS-IOT-UNSA SUSCRIBOR | fw 1.1 ===");
  pinMode(2, OUTPUT);

  WiFi.mode(WIFI_STA); WiFi.setSleep(false);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(250); Serial.print("."); }
  Serial.printf("\n[WiFi] OK IP=%s canal=%d\n",
                WiFi.localIP().toString().c_str(), WiFi.channel());
  configTime(0, 0, NTP_SERVER); delay(1500);

  mqtt.setServer(MQTT_HOST, 1883);
```

```
  mqtt.setClientId("suscriptor");
  mqtt.setWill("tesis/suscriptor/estado", 0, true, "off");
  mqtt.onConnect(onMqttConnect);
  mqtt.onDisconnect(onMqttDisconnect);
  mqtt.onMessage(onMqttMessage);
  mqtt.connect();
}

void loop() {
  if (!mqtt.connected()) mqtt.connect();
  if (recibidos > 0 && recibidos % 200 == 0)
    Serial.printf("[STAT] rx=%llu dup=%llu\n", recibidos, duplicados);
}
```

Pérdida de paquetes: el suscriptor no la adivina — el *gap* de `seq` por nodo (`seq_max − únicos_recibidos`) y la comparación con el log de Mosquitto (§4.1) la determinan en la PC; con ello se calcula la PDR de cada celda.

3.5 Matriz de configuración B y C

Factor	Nivel	Valor en firmware	Envío por
B1 esparsa	intervalo 5000 ms	`#define B1_MS` o `b=5000`	`tesis/control/config`
B2 moderado	intervalo 1000 ms	`b=1000`	idem
B3 intenso	intervalo 250 ms	`b=250`	idem
C1/C2/C3	QoS 0/1/2	`qos=0|1|2`	idem

Los valores exactos de B se fijan **antes** de la campaña y se reportan en el capítulo de

resultados (§3.1.5). Tras fijarlos no se modifican.

3.6 Prueba en mesa (antes de salir al escenario)

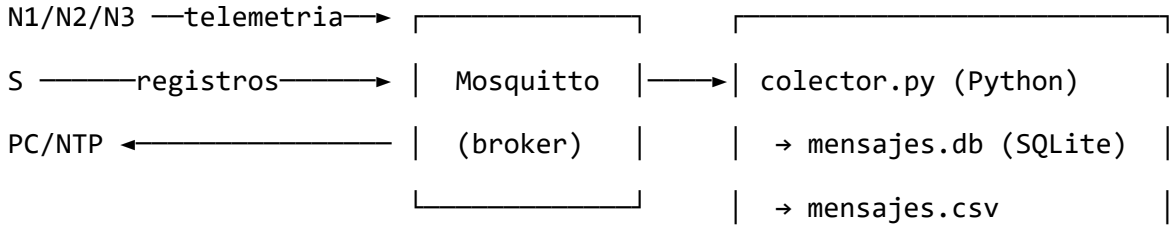
```
# 1. Consola serie: banner con IP, canal y epoch correctos
# 2. Orden manual:
mosquitto_pub -h 192.168.1.100 -t tesis/control/config -m "b=1000;qos=1;c=A1B2C2;rep=0" -q 1
mosquitto_pub -h 192.168.1.100 -t tesis/control/orden  -m "iniciar" -q 1
# 3. Ver flujo:
mosquitto_sub -h 192.168.1.100 -t "tesis/#" -v | head -20
# 4. Parar:
mosquitto_pub -h 192.168.1.100 -t tesis/control/orden -m "parar" -q 1
```

Checklist Fase 3

- ☐ Compilación sin errores para las 4 variantes (`NODE_ID` distinto por binario).
- ☐ Misma versión de firmware en las 4 placas (hash/versión impresa en banner).
- ☐ `epoch` NTP válido en los 4 nodos.
- ☐ QoS efectivo del tópico `telemetria` verificado en red para 0/1/2.
- ☐ Latencia calculada por el suscriptor coincide ±2 ms con el cálculo en la PC.

Fase 4 — Adquisición y almacenamiento de datos

4.1 Arquitectura del colector



```
                                        │  → ensayos.csv (resumen) │
                                        └──────────────────────────┘
```

4.2 Geometría fija (`geometría.csv`)

```
nodo,escenario,distancia_m,muros_concreto
N1,A1,3.0,0
N2,A2,5.0,1
N3,A3,8.0,2
S,A1,3.0,0
S,A2,5.0,1
S,A3,8.0,2
```

(Completar con las medidas reales del croquis §1.4. Este archivo aporta la variable *distancia* del modelo, §5.1.)

4.3 Script de suscripción y almacenamiento (`colector.py`)

```python
#!/usr/bin/env python3
"""
colector.py — tesis-iot-unsa
Suscripción al broker Mosquitto y almacenamiento persistente de las
variables en dataset tabular (SQLite + CSV). Ejecutar en la PC de
adquisición mientras un ensayo está corriendo.
"""
import json, csv, sqlite3, time, threading, sys, argparse
from datetime import datetime, timezone
import paho.mqtt.client as mqtt


BROKER = "127.0.0.1"
```

```
PORT   = 1883
DB     = "mensajes.db"

# Compatibilidad paho 1.x / 2.x
try:
    cli = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
except AttributeError:
    cli = mqtt.Client()

con = sqlite3.connect(DB, check_same_thread=False)
cur = con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS mensajes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  ensayo TEXT, nodo TEXT, seq INTEGER,
  t_tx_ms INTEGER, t_rx_ms INTEGER, lat_ms REAL,
  rssi INTEGER, noise_floor INTEGER, celda TEXT, replica INTEGER,
  q INTEGER, t_srv TEXT, tipo TEXT)""")
cur.execute("CREATE INDEX IF NOT EXISTS ix_msg ON mensajes(ensayo, nodo, seq)")
con.commit()

ensayo_activo = {"id": None, "celda": None, "rep": None, "q": None, "inicio": None}
lock = threading.Lock()

def guardar(fila):
    with lock:
        cur.execute("""INSERT INTO mensajes
```

```
        (ensayo,nodo,seq,t_tx_ms,t_rx_ms,lat_ms,rssi,noise_floor,celda,replica,q,t_srv,tipo)
           VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)""", fila)
        con.commit()

def armar(tipo, dato):
    ahora = int(time.time() * 1000)
  n = dato.get("n"); s = dato.get("s"); celda = dato.get("c") or ensayo_activo["celda"]
  tx = dato.get("tx"); lat = dato.get("lat"); r = dato.get("r"); nf = dato.get("nf")
    with lock:
        e = ensayo_activo
        if e["id"] is None:            # fuera de ensayo: se ignora
            return
        guardar((e["id"], n, s, tx, ahora, lat, r, nf, celda,
                 e["rep"], e["q"],
           datetime.now(timezone.utc).isoformat(timespec="milliseconds"), tipo))

def on_connect(cli, userdata, flags, reason_code, properties=None):
    rc = getattr(reason_code, "value", reason_code)
    print(f"[conectado rc={rc}]")
    cli.subscribe([("tesis/nodos/+/telemetria", 1),
                   ("tesis/suscriptor/registros", 1),
                   ("tesis/nodos/+/estado", 0)])

def on_message(cli, userdata, msg):
    try:
        dato = json.loads(msg.payload.decode())
    except Exception as ex:
```

```
        print("[JSON ERROR]", ex); return
    if msg.topic.endswith("/telemetria"):      armar("tx", dato)
    elif msg.topic.endswith("/registros"):     armar("rx", dato)
    else: print("[estado]", msg.topic, msg.payload.decode())

cli.on_connect = on_connect
cli.on_message = on_message

def iniciar_ensayo(ensayo_id, celda, replica, q):
    with lock:
        ensayo_activo.update(id=ensayo_id, celda=celda, rep=replica, q=q,
                             inicio=int(time.time() * 1000))
    print(f"[ensayo] {ensayo_id} celda={celda} rep={replica} q={q} INICIADO")

def detener_ensayo():
    with lock:
        eid = ensayo_activo["id"]
       ensayo_activo.update(id=None, celda=None, rep=None, q=None, inicio=None)
    if eid:
        cur.execute("""INSERT INTO ensayos (ensayo, fin_ms) VALUES (?,?)
                    ON CONFLICT(ensayo) DO UPDATE SET fin_ms=excluded.fin_ms""",
                     (eid, int(time.time() * 1000)))
        con.commit()
    print(f"[ensayo] {eid} DETENIDO")

cur.execute("""CREATE TABLE IF NOT EXISTS ensayos
               (ensayo TEXT PRIMARY KEY, fin_ms INTEGER)""")
```

```
con.commit()


if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--ensayo", help="id p.ej. A2B3C2_r1")
    p.add_argument("--celda");  p.add_argument("--rep", type=int)
    p.add_argument("--q", type=int, default=1)
    a = p.parse_args()
    if a.ensayo: iniciar_ensayo(a.ensayo, a.celda, a.rep, a.q)
    cli.connect(BROKER, PORT, 60)
    cli.loop_forever()
```

Uso en modo síncrono (un ensayo por proceso):

```
pip install paho-mqtt pandas scikit-learn scipy matplotlib joblib
python colector.py --ensayo A2B3C2_r1 --celda A2B3C2 --rep 1 --q 2
# ... dejar correr 10 minutos (o usar el orquestador §4.5) ...
# Ctrl+C -> detención y cierre
```

Exportar a CSV:

```
python -c "import sqlite3,pandas as pd; c=sqlite3.connect('mensajes.db');
pd.read_sql('SELECT * FROM mensajes', c).to_csv('mensajes.csv', index=False)"
```

4.4 Protocolo operativo de las 27 celdas (3×3×3)

Combinaciones: `A{1,2,3} × B{1,2,3} × C{1,2,3}` = **27 celdas**; **3 réplicas** cada una = **81 ensayos**; cada ensayo = **10 minutos**.

```
# generar_secuencia.py — orden aleatorizado ANTES de la campaña (§3.1.5)
import random, csv
celdas = [f"A{a}B{b}C{c}" for a in (1,2,3) for b in (1,2,3) for c in (1,2,3)]
ensayos = [(cel, rep) for cel in celdas for rep in (1,2,3)]
```

```
random.Random(42).shuffle(ensayos)          # semilla fija y registrada
with open("secuencia_81.csv", "w", newline="", encoding="utf-8") as f:
   w = csv.writer(f); w.writerow(["orden","celda","replica","qos","b_ms","id","inicio","fin"
    for i, (cel, rep) in enumerate(ensayos, 1):
        w.writerow([i, cel, rep, int(cel[5]), [5000,1000,250][int(cel[3])-1],
                    f"{cel}_r{rep}", "", "", "", ""])
print("81 ensayos generados. NO reordenar por conveniencia operativa.")
```

Secuencia por ensayo (checklist operativo):

1. Verificar condiciones fijas: canal Wi-Fi (foto/panel), NTP ≤ ±10 ms en 4 nodos, broker activo, ambiente conforme a §3.1.2 (puertas, ocupación, electrodomésticos).
2. Ubicar S según el nivel **A** de la celda (fijar con cinta; medir distancia).
3. Iniciar colector con id de ensayo (`--ensayo A2B3C2_r1 ...`).
4. Enviar configuración de celda y orden:

   ```
   mosquitto_pub -t tesis/control/config -m "b=250;qos=2;c=A3B3C3;rep=1" -q 1
   mosquitto_pub -t tesis/control/orden  -m "iniciar" -q 1
   ```
5. Dejar **600 s** exactos; no tocar puertas/equipos; registrar incidencias.
6. Al vencido: `mosquitto_pub -t tesis/control/orden -m "parar" -q 1` y detener colector.
7. Verificar volumen esperado de mensajes (≈ `600/intervalo × 3 nodos × (1−pérdida esperada)`); si es < 90 % → revisar y, según condiciones de parada, **anular y reejecutar el ensayo íntegro**.
8. Asentar en el cuaderno: hora inicio/fin, desfase NTP, variante de A3 empleada, incidentes.
9. Continuar con el **siguiente orden** de `secuencia_81.csv` (prohibido reordenar).

Condiciones de parada (reproducidas de §3.1.5): reinicio del router; pérdida de NTP; caída de broker/BD; alteración ambiental no programada → anular ensayo, documentar, reprogramar íntegro en la primera ventana disponible.

4.5 Orquestador opcional de 10 minutos

```
# orquestador.py — cronómetro de ensayo + comandos MQTT
import time, subprocess, sys, json
dur = 600  # segundos
cfg, eid, rep, q = sys.argv[1], sys.argv[2], int(sys.argv[3]), int(sys.argv[4])
pub = lambda t, m: subprocess.run(["mosquitto_pub", "-t", t, "-m", m, "-q", "1"])
pub("tesis/control/config", f"b={cfg};qos={q};c={eid.split('_')[0]};rep={rep}")
pub("tesis/control/orden", "iniciar")
time.sleep(dur)
pub("tesis/control/orden", "parar")
print(f"{eid} completado: {dur} s")
```

Uso (desde la PC, con `secuencia_81.csv` como fuente de verdad):

```
python orquestador.py 250 A3B3C3_r1 1 2
#   args: b_ms celda_replica replica qos     -> 600 s exactos y orden parar
```

4.6 Estructura final del dataset

```
tesis-iot-unsa/
├── datos/
│   ├── mensajes.db        # registro crudo (triangulado tx/rx)
│   ├── mensajes.csv       # export
│   ├── secuencia_81.csv   # orden aleatorizado (evidencia)
│   ├── geometria.csv      # distancias por nodo/escenario
│   └── ensayos.csv      # resumen por ensayo: PDR, lat, jitter, pérdida, buen uso
└── modelos/               # salida de la Fase 5
```

Columnas de `mensajes.csv`: `ensayo, nodo, seq, t_tx_ms, t_rx_ms, lat_ms, rssi, noise_floor, celda, replica, q, t_srv, tipo`.

Checklist Fase 4

- ☐ Secuencia de 81 ensayos generada con semilla y archivada.
- ☐ `mensajes.db` crece durante la prueba piloto de 2 min (≥ 100 filas).
- ☐ CSV exportado y legible por pandas sin codificación rota.
- ☐ Cuaderno de campo con columnas: orden, inicio, fin, NTP, ambiental, incidente, anulado-sí/no.

Fase 5 — Análisis con Machine Learning

Alineado con `metodologia.md` §3.4.5: pipeline sin fuga, split 70/30 semilla 42, CV 10 pliegues, 4 algoritmos por vertiente, criterios 85 % (clasificación) y RMSE ≤ 15 % del rango (regresión).

5.1 Preprocesamiento y limpieza

```python
# 01_preparar.py
import pandas as pd, numpy as np

df = pd.read_csv("datos/mensajes.csv")

# --- Limpieza básica ---
df = df.drop_duplicates(subset=["ensayo", "nodo", "seq"])     # duplicados exactos
df = df[df["lat_ms"].between(-5, 10000)]                  # trampas de reloj/NTP
df = df[df["rssi"].between(-100, -20)]                   # rango físico del chip
df["snr_db"] = df["rssi"] - df["noise_floor"]                    # derivada

# --- Etiquetas de factor -> ordinales (conservan el orden lógico) ---
df["A"] = df["celda"].str[1].astype(int)
df["B"] = df["celda"].str[3].astype(int)
```

```
df["C"] = df["q"].astype(int)

# --- Distancia desde geometría (el escenario A se deriva de la celda) ---
df["escenario"] = "A" + df["celda"].str[1]
geo = pd.read_csv("datos/geometria.csv")
df = df.merge(geo, on=["nodo", "escenario"], how="left")

# --- Respuestas por celda/ensayo (agregación de referencia) ---
def pdr(g):
    esp = g["seq"].max(); rec = g["seq"].nunique()
    return 100.0 * rec / esp if esp else np.nan

resumen = (df.groupby(["ensayo", "celda"])
             .apply(lambda g: pd.Series({
                 "pdr": pdr(g),
                 "lat_media": g["lat_ms"].mean(),
                 "lat_p95": np.percentile(g["lat_ms"], 95),
                 "jitter": g["lat_ms"].std(),
                 "perdida_pct": 100 - pdr(g)}))
             .reset_index())
resumen["pdr_ok"] = (resumen["pdr"] >= 85).astype(int)          # umbral §3.5.1

# --- Target a nivel mensaje (clasificación hereda la etiqueta de su celda) ---
df = df.merge(resumen[["ensayo", "pdr", "pdr_ok"]], on="ensayo")
df.to_csv("datos/dataset_ml.csv", index=False)
resumen.to_csv("datos/ensayos.csv", index=False)
print(df.shape, "\nBalance de clases:\n", df["pdr_ok"].value_counts(normalize=True))
```

Reglas del pipeline (§3.4.5):

Paso	Método	Cuándo aplica
Imputación de faltantes	`SimpleImputer(strategy="median")`	Siempre; ajustar **solo** sobre train
Categóricas ordenadas	`OrdinalEncoder` (A, B, C conservan orden)	Siempre
Escalado	`StandardScaler`	Solo LR logística y Ridge (árbol/bosque no lo requieren)
Balanceo	`class_weight="balanced"` o re-muestreo	Si `pdr_ok` desbalanceado > 60/40
Sin fuga	todo dentro de `Pipeline` + `GridSearchCV`	Siempre

5.2 Entrenamiento (clasificación y regresión)

```
# 02_entrenar.py
import pandas as pd, numpy as np, joblib
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix, mean_absolute_error,
```

```
    mean_squared_error, r2_score)

df = pd.read_csv("datos/dataset_ml.csv")
feats = ["rssi", "snr_db", "distancia_m", "A", "B", "C"]
X = df[feats]
num = ["rssi", "snr_db", "distancia_m"]; cat = ["A", "B", "C"]

pre_esc  = Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc",  StandardScaler())])
pre_base = Pipeline([("imp", SimpleImputer(strategy="median"))])
pre = ColumnTransformer([("num", pre_esc, num), ("cat", "passthrough", cat)])

Xtr, Xte, ytr, yte = train_test_split(
    X, df["pdr_ok"], test_size=0.30, random_state=42, stratify=df["pdr_ok"])

# ---------- VERTIENTE DE CLASIFICACIÓN ----------
clfs = {
 "logistica": (LogisticRegression(max_iter=1000, class_weight="balanced"),
               {"clf__C": [0.1, 1, 10]}),
 "arbol":     (DecisionTreeClassifier(random_state=42),
           {"clf__max_depth": [4, 8, None], "clf__min_samples_leaf": [5, 20]}),
 "bosque":    (RandomForestClassifier(random_state=42, n_jobs=-1),
               {"clf__n_estimators": [200], "clf__max_depth": [8, 16, None]}),
}
mejor, nombre, score = None, None, -1
for k, (modelo, grid) in clfs.items():
    pipe = Pipeline([("pre", pre), ("clf", modelo)])
```

```
    gs = GridSearchCV(pipe, grid, cv=10, scoring="f1", n_jobs=-1)
    gs.fit(Xtr, ytr)
    print(f"[{k}] F1(CV)={gs.best_score_:.3f}  {gs.best_params_}")
    if gs.best_score_ > score: mejor, nombre, score = gs, k, gs.best_score_

y_pred = mejor.predict(Xte)
print("\n=== CLASIFICACIÓN (holdout 30 %) ===")
print(f"Modelo ganador: {nombre}")
print(f"Accuracy = {accuracy_score(yte, y_pred):.3f} | "
      f"F1 = {f1_score(yte, y_pred):.3f} | "
      f"Precision = {precision_score(yte, y_pred, zero_division=0):.3f} | "
      f"Recall = {recall_score(yte, y_pred):.3f}")
print("Matriz de confusión [VN FP / FN VP]:")
print(confusion_matrix(yte, y_pred))
try:
   auc = roc_auc_score(yte, mejor.predict_proba(Xte)[:, 1]); print(f"AUC-ROC = {auc:.3f}")
except Exception: pass
print(f"Criterio 85 %: {'CUMPLE' if score >= 0.85 else 'NO CUMPLE -> contingencia §3.5.2'}")
joblib.dump(mejor, "modelos/clasificador.joblib")

# ---------- VERTIENTE DE REGRESIÓN (latencia, jitter) ----------
Xtr_r, Xte_r, ytr_r, yte_r = train_test_split(
    X, df["lat_ms"], test_size=0.30, random_state=42)
reg = Pipeline([("pre", pre), ("reg", Ridge())])
reg.fit(Xtr_r, ytr_r)
yp = reg.predict(Xte_r)
rmse = float(np.sqrt(mean_squared_error(yte_r, yp)))  # compatible con toda versión de scikit
```

```
rango = yte_r.max() - yte_r.min()
print("\n=== REGRESIÓN (latencia ms) ===")
print(f"MAE={mean_absolute_error(yte_r, yp):.2f} | RMSE={rmse:.2f} | "
      f"R2={r2_score(yte_r, yp):.3f}")
print(f"RMSE / rango = {100*rmse/rango:.1f} %  (criterio ≤ 15 %)")
joblib.dump(reg, "modelos/regresion_latencia.joblib")
```

5.3 Métricas de evaluación y criterios

Vertiente	Métrica	Interpretación operativa	Criterio de aceptación
Clasificación	Exactitud, F1 (CV 10 pliegues)	Capacidad de anticipar incumplimiento del 85 %	**≥ 85 %**
Clasificación	Precisión / Recall	FP = alerta falsa de degradación; FN = degradación no detectada	Reportar siempre
Clasificación	Matriz de confusión	Distribución VN/FP/FN/VP en tabla 2×2	Obligatoria
Clasificación	AUC-ROC	Discriminación umbral-independent	Reportar
Regresión	**RMSE (ms)**	Error típico de predicción de latencia	**≤ 15 % del rango observado**

Vertiente	Métrica	Interpretación operativa	Criterio de aceptación
Regresión	MAE, MAPE, R^2	Error robusto / varianza explicada	Reportar

Ramificación (§4.6 de la tesis): si ninguno de los dos criterios se alcanza → activar pasos 3–5 del plan de contingencia (más réplicas, revisión de parámetros configurables, informe de desviación). Nunca redefinir el umbral sobre datos ya recolectados.

5.4 Inferencia y serialización

```
import joblib, pandas as pd
clf = joblib.load("modelos/clasificador.joblib")
nuevo = pd.DataFrame([{"rssi": -72, "snr_db": 20, "distancia_m": 5.0,
                       "A": 2, "B": 3, "C": 2}])
print("Predicción (1 = cumple PDR ≥ 85 %):", clf.predict(nuevo)[0])
```

Checklist Fase 5

- ☐ `dataset_ml.csv` sin nulos inesperados; `pdr` rango [0, 100].
- ☐ Split reproducido dos veces con `random_state=42` → idéntico.
- ☐ Ningún escalador/imputador ajustado sobre el holdout (verificado por `Pipeline`).
- ☐ Tabla de modelos comparados (§3.4.5) poblada con F1/RMSE reales.
- ☐ Matriz de confusión 2×2 con los cuatro valores reportada en resultados.
- ☐ Criterios 85 % / 15 % evaluados y decisión documentada (cumple → resultados; no cumple → contingencia).

Lista de verificación global

#	Hito	Evidencia
1	4 nodos armados, alimentación estable 15 min	Fotos + banner serie
2	Croquis medido, `geometría.csv` completo	CSV + fotos por nivel A
3	Router en canal fijo 1/6/11, 20 MHz, WAN caída	Foto del panel
4	Mosquitto como servicio + log con ms	`mosquitto_sub -t "tesis/#"`
5	NTP local ≤ ±10 ms en 4 nodos	Lectura banner
6	Firmware idéntico en 4 placas; QoS 0/1/2 verificado	Versión + captura red
7	Prueba piloto 2 min → filas en `mensajes.db`	SQLite count
8	Secuencia de 81 ensayos generada y archivada	`secuencia_81.csv`
9	81 ensayos completados sin violar condiciones de parada	Cuaderno de campo
10	`mensajes.csv` + `ensayos.csv` + `dataset_ml.csv` (§5.1 los genera)	CSVs
11	Modelos entrenados, métricas y matrices emitidas	`modelos/*.joblib` + log

#	Hito	Evidencia
12	Criterios 85 % / 15 % evaluados	Tabla §5.3 en resultados

Regla rectora: la integridad del protocolo prevalece sobre el cumplimiento del criterio — ensayos anulados se reejecutan íntegros, datos no se editan retroactivamente y todo incidente queda en el cuaderno de campo.